

Real-Time Monte Carlo Localization

for TurtleBot3 in Gazebo + ROS 2

MATLAB Implementation — Technical README

1. Overview

This MATLAB script implements a real-time Monte Carlo Localization (MCL) pipeline for a TurtleBot3 mobile robot operating inside the Gazebo simulation environment. The algorithm fuses odometry and LiDAR sensor data to estimate the robot's 2-D pose (x , y , heading θ) on a pre-built occupancy grid map. Communication with the simulator is handled entirely over ROS 2 using MATLAB's Robotics System Toolbox.

MCL — also known as the particle filter — maintains a probabilistic belief over all possible robot positions by propagating a set of weighted particles through a motion model and re-weighting them with a sensor model. Over time the particle cloud converges onto the true robot pose.

2. Key Features

- Full ROS 2 integration via Fast DDS Discovery Server
- Pre-loaded occupancy grid map support (.mat file)
- Odometry motion model with configurable Gaussian noise
- Likelihood-field LiDAR sensor model
- Global localization — no initial pose prior required
- Adaptive particle count (500 – 5 000 particles)
- Real-time animated visualization of particles and estimated pose
- 10 Hz control loop managed by `ros2rate`
- Robust handling of Inf/NaN values in LiDAR range data

3. Prerequisites

3.1 Software

Component	Version / Notes
MATLAB	R2022b or later recommended
Robotics System Toolbox	Required for MCL, odometryMotionModel, lidarScan, ros2node etc.
Navigation Toolbox	Required for likelihoodFieldSensorModel
ROS 2	Humble Hawksbill (or compatible) installed on the host or a reachable machine
Gazebo	Gazebo Classic 11 or Ignition; TurtleBot3 packages must be sourced
Fast DDS	rmw_fastrtps_cpp middleware; Discovery Server enabled

3.2 Required Files

File	Purpose
montecarlo.m	Main MATLAB script (this file)
turtletword_grid_occupancy.mat	Pre-built occupancy grid map of the Gazebo world; must contain a variable named myOccMap of type occupancyMap

3.3 ROS 2 Topics

Topic	Message Type	Direction	Usage
/odom	nav_msgs/Odometry	Subscribe	Robot pose & orientation from wheel encoders
/scan	sensor_msgs/LaserScan	Subscribe	360° LiDAR range measurements

4. Configuration

4.1 Network / ROS 2 Environment Variables

These are set programmatically at the top of the script using setenv():

Variable	Default Value	Description
ROS_DOMAIN_ID	0	ROS 2 domain; must match the robot's domain
RMW_IMPLEMENTATION	rmw_fastrtps_cpp	Selects Fast DDS as the ROS 2 middleware
ROS_DISCOVERY_SERVER	192.168.1.84:11811	IP:port of the Fast DDS Discovery Server running on the robot or server PC

⚠ Update `ROS_DISCOVERY_SERVER` to the actual IP address of your Discovery Server before running.

4.2 Odometry Motion Model

Object type: `odometryMotionModel`

Parameter	Value	Description
Noise	[0.2, 0.2, 0.2, 0.2]	Four-element vector [α_1 α_2 α_3 α_4] controlling Gaussian noise on rotation-from-rotation, rotation-from-translation, translation-from-translation, and translation-from-rotation respectively

Larger noise values widen the particle spread and help recover from odometry drift at the cost of convergence speed.

4.3 Likelihood-Field Sensor Model

Object type: `likelihoodFieldSensorModel`

Parameter	Value	Description
Map	myOccMap	Occupancy grid map loaded from the .mat file
SensorLimits	[0.12, 3.5]	Min/max valid LiDAR range in metres; readings outside this band are discarded

4.4 MCL Object Parameters

Object type: `monteCarloLocalization`

Property	Value	Description
UseLidarScan	true	Enables lidarScan object input (as opposed to raw range arrays)
MotionModel	odometryModel	Assigns the configured odometry motion model
SensorModel	sensorModel	Assigns the configured likelihood-field sensor model
UpdateThresholds	[0.1, 0.1, 0.1]	Minimum [Δx (m), Δy (m), $\Delta \theta$ (rad)] motion before the filter updates; prevents unnecessary computation when the robot is stationary
ResamplingInterval	1	Resample particles every N updates; 1 means resample on every filter update
ParticleLimits	[500, 5000]	Adaptive particle count range [min, max]; KLD-sampling adjusts the count based on the estimated uncertainty

Property	Value	Description
GlobalLocalization	true	Distributes particles uniformly over the entire free space — no initial pose estimate required. Set to false if the starting pose is known to accelerate convergence

5. Script Architecture

The script is structured in five sequential stages:

Stage 1 — Environment Setup (Lines 1–17)

Clears the MATLAB workspace, sets ROS 2 environment variables, and attempts to create a named ROS 2 node (/matlab_mcl_node). A try/catch block gracefully handles the case where the node already exists.

Stage 2 — Map Loading (Lines 19–22)

Loads the occupancy grid map from turtletword_grid_occupancy.mat. The file must expose a variable named myOccMap. The map is passed directly into the sensor model, so its coordinate frame and resolution define the localization reference.

Stage 3 — Model & MCL Initialization (Lines 24–47)

Creates and configures:

- odometryMotionModel — propagates particles according to wheel-encoder odometry
- likelihoodFieldSensorModel — scores particles by comparing expected vs. actual laser hits on the map
- monteCarloLocalization — the main filter object that ties both models together

Stage 4 — ROS 2 Subscriber Setup & Warm-Up (Lines 49–60)

Creates two subscribers for /odom and /scan, then blocks (up to 10 seconds each) waiting for the first messages to arrive. This warm-up guarantees valid data before the main loop begins.

Stage 5 — Real-Time Processing Loop (Lines 62–end)

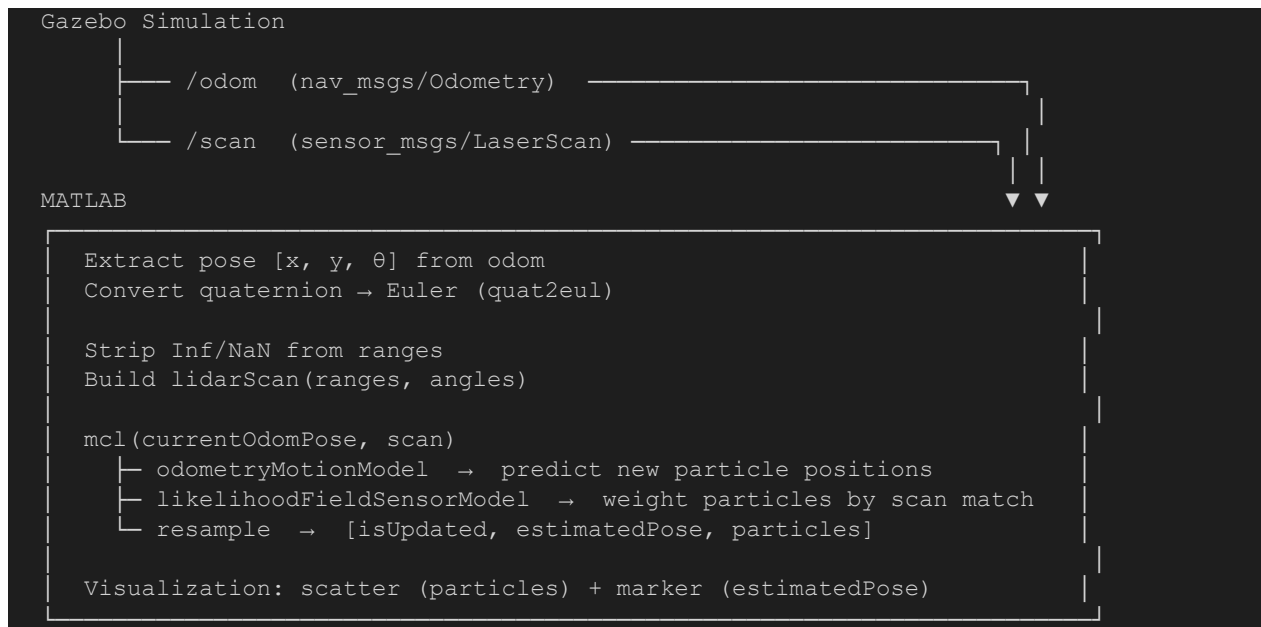
A while true loop runs at 10 Hz (ros2rate). Each iteration:

- Reads the latest odometry and LiDAR messages
- Skips the iteration if either message is empty
- Converts the quaternion orientation to Euler angles (yaw extracted as θ)
- Strips Inf and NaN values from the LiDAR range array

- Calls the MCL object: `[isUpdated, estimatedPose, particles] = mcl(currentOdomPose, scan)`
- If the filter updated, refreshes the scatter plot (particles) and pose marker on the occupancy map
- Calls `waitfor(rate)` to sleep for the remainder of the 100 ms period

6. Data Flow Diagram

Below is the end-to-end data path through the system:



7. Visualization

A MATLAB figure window titled Monte Carlo Localization is created before the main loop. It contains:

Plot Element	Object Type	Colour	Meaning
Occupancy map	<code>show(myOccMap)</code>	Grey-scale	Static background; white = free, black = occupied, grey = unknown
Particle cloud	<code>scatter (size 5)</code>	Red	Current belief distribution; each dot is one particle [x, y]
Estimated pose	<code>plot marker</code>	Green circle	MAP/mean pose estimate output by the MCL filter

drawnow limitrate is used instead of drawnow to cap the rendering rate and prevent the visualizer from slowing down the control loop.

8. Running the Script

Step 1 — Start the Gazebo Simulation

```
# In a terminal with ROS 2 sourced:  
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

Step 2 — Start the Fast DDS Discovery Server

```
fastdds discovery -i 0 -p 11811
```

Run this on the machine whose IP matches ROS_DISCOVERY_SERVER in the script.

Step 3 — Place the Map File

Copy turtletword_grid_occupancy.mat into your MATLAB working directory (or add its folder to the MATLAB path).

Step 4 — Run the Script

```
>> montecarlo
```

MATLAB will print status messages to the Command Window:

```
ROS 2 Node created successfully.  
subscribe to the ros2 topics  
Waiting for sensor data from gazebo simulation environment  
Data received! Starting Monte carlo localization
```

The MCL figure window will open and particles will be visible almost immediately. If GlobalLocalization = true the particles start spread across the whole map and converge as the robot moves.

9. Known Bug Fix — Particle Access

⚠ This section documents a critical correction applied to the original code.

In recent versions of MATLAB's Robotics System Toolbox, monteCarloLocalization does NOT expose a public .Particles property. Attempting to read mcl.Particles throws:

```
Unrecognized method, property, or field 'Particles'
for class 'monteCarloLocalization'.
```

The fix is to capture the particle array as the third return value of the MCL step call:

```
% ✗ WRONG — throws runtime error
[isUpdated, estimatedPose, ~] = mcl(currentOdomPose, scan);
particles = mcl.Particles;
```

```
% ✓ CORRECT — particles returned directly
[isUpdated, estimatedPose, particles] = mcl(currentOdomPose, scan);
```

The third output is an $N \times 3$ matrix where each row contains $[x, y, \theta]$ for one particle. The visualization uses columns 1 and 2 (x, y) for the scatter plot.

10. Tuning Guide

Symptom	Likely Cause	Suggested Fix
Particles never converge	Noise too high or sensor model mismatch	Reduce <code>odometryModel.Noise</code> ; verify map matches the simulated world
Filter loses track after turning	Rotation noise too low	Increase α_1 and α_4 in the Noise vector
Very slow convergence from global init	Too few particles or tight limits	Increase <code>ParticleLimits</code> upper bound (e.g. [500 10000])
CPU usage too high	Too many particles or update rate too high	Lower the <code>ros2rate</code> frequency or reduce <code>ParticleLimits</code> upper bound
Estimated pose jumps erratically	<code>UpdateThresholds</code> too small	Increase thresholds, e.g. [0.2, 0.2, 0.05]
Particles cluster in wrong room	Map origin mismatch	Verify <code>myOccMap</code> coordinate frame matches /odom frame
No data received (timeout)	Discovery Server unreachable	Confirm IP in <code>ROS_DISCOVERY_SERVER</code> and that <code>fastdds</code> discovery is running

11. Limitations & Future Work

- 2-D only — the algorithm assumes a planar environment; 3-D localization is not supported.

- Static map — the occupancy grid is loaded once at start-up; dynamic obstacles are not modelled.
- No loop-closure — large environments with repeated structure may cause ambiguity.
- Single-threaded — ROS 2 callbacks and the MCL update share the same MATLAB thread; high LiDAR rates may cause the loop to fall behind.
- No pose-graph back-end — accumulated odometry drift is not corrected beyond the particle filter.

Potential improvements include saving the estimated trajectory to a ROS 2 bag or CSV file, publishing the estimated pose as a `/initialpose` message for downstream navigation, and integrating a SLAM back-end for simultaneous mapping.

12. Variable & Object Reference

Variable	Type	Description
node	ros2node	ROS 2 node handle (<code>/matlab_mcl_node</code>)
myOccMap	occupancyMap	Pre-built grid map of the Gazebo world
odometryModel	odometryMotionModel	Gaussian noise motion model
sensorModel	likelihoodFieldSensorModel	Likelihood-field laser scoring model
mcl	monteCarloLocalization	Main particle filter object
odomSub	ros2subscriber	Subscriber to <code>/odom</code> (<code>nav_msgs/Odometry</code>)
scanSub	ros2subscriber	Subscriber to <code>/scan</code> (<code>sensor_msgs/LaserScan</code>)
rate	ros2rate	10 Hz loop-rate controller
currentOdomPose	1×3 double	[<i>x</i> , <i>y</i> , <i>θ</i>] pose extracted from the latest odometry message
scan	lidarScan	Structured laser scan object passed to the MCL filter
isUpdated	logical	True when the MCL filter performed an update step
estimatedPose	1×3 double	[<i>x</i> , <i>y</i> , <i>θ</i>] MAP pose estimate from the filter
particles	N×3 double	All particle states [<i>x</i> , <i>y</i> , <i>θ</i>]; <i>N</i> varies between <code>ParticleLimits</code>
plotParticles	scatter handle	Graphics handle for the live particle scatter plot
plotEstimatedPose	line handle	Graphics handle for the estimated pose marker